

RADAR-TASYTRADE — Plano Mestre de Remediação

Da fabricação de dados ao “só dados reais”: estado atual, frentes de trabalho, riscos e sequência recomendada

Preparado para Duda

08 de setembro de 2026

0. Critério de sucesso e por que este plano existe

Cinco ciclos de auditoria (interna e independente) elevaram a nota de integridade de dado de 2,8/10 para 5,3/10 — e ainda assim uma verificação visual manual sua, contra a plataforma real da Tastytrade, achou em minutos um erro que nenhum dos cinco ciclos pegou: a recomendação de BAC ofereceu um strike (\$47,50) que não existe e um vencimento (35 DTE) que não é o real (38 DTE).

Isso não é acidente de um ciclo específico — é sintoma de uma lacuna estrutural no processo até aqui: todo ciclo auditou **correção matemática de fórmula** (BSM bate com cálculo independente, GEX bate com cálculo manual) e nenhum auditou **plausibilidade contra a grade real de mercado**. Fórmula pode estar certa e ainda assim descrever um mundo que não existe, se o insumo (strike, vencimento, cotação) for sintético.

Sua diretriz mudou o critério de aceite: **daqui em diante, só dado real**. Este plano parte disso — não é mais “documentar limitação e seguir”, é “sem fonte real confirmada, a funcionalidade declara indisponibilidade, não fabrica valor”.

Critério de “pronto” para qualquer frente abaixo: passa em uma auditoria cega independente (sessão nova, sem meu contexto) que confirme, contra a API real ou contra a plataforma, que os números batem — não que a fórmula está certa.

1. Estado atual (fatos confirmados, não opinião)

1.1 O que está confirmado correto e não precisa ser mexido

- Motor BSM** (bsm-pricer.ts): validado de forma independente (Ciclo 5) contra cálculo Python/scipy equivalente e paridade put-call exata.
- Motor GEX** (gex-engine.ts): fórmula $\Gamma \times 0I \times Spot^2 \times 100 \times 0,01 / 1.000.000$ confirmada correta por cálculo manual e pelo auditor independente.
- Segurança de API** (api-guard.ts): rate limit em 3 camadas, validação de origem — implementado e documentado nesta sessão, sem pendência conhecida.
- Autenticação Tastytrade real** (tastytrade-auth.service.ts + getOptionChain() novo): confirmado nesta sessão contra a API real — 19 vencimentos, strikes inteiros reais para BAC, batendo com o que você viu na plataforma.

Não gaste ciclo de revisão nisso de novo. O núcleo matemático está provado; o problema é o que alimenta esse núcleo.

1.2 O que está confirmado fabricado ou pendente

Item	Local	Natureza do problema	Origem da confirmação
Strike/DTE de opções	volatility-engine.ts (step , dte fixos)	Strike e vencimento sintéticos, sem checar existência real	Seu achado visual (BAC)

Preço/grego das pernas	volatility-engine.ts (BSM com multiplicadores de IV arbitrários: $\times 1.08$, $\times 0.98$...)	Mesmo com strike/DTE reais, o prêmio ainda é modelo, não cotação	Análise desta sessão
Checklist fixo da tela de cotação	QuoteView.tsx:317-323	"SPOT: REAL", "NET GEX POSITIVO", "FED 4.50%" fixos, não recalculados por ticker	Achado C5-01, Ciclo 5
Fundamentos fabricados (2ª ocorrência)	ai-consultant.ts:74-76	currentRatio: 1.45, ebitdaMargin: 0.28 fixos — mesmo padrão já corrigido em QuoteView.tsx, mas não replicado aqui	Achado C5-04, Ciclo 5
Infra de proveniência órfã	provenance.ts	ProvenanceBadge / combineProvenance() existem, zero import em src/	Achado C5-07, Ciclo 5
Fundamentos de tickers US	Sem fonte	Sem vendedor de fundamentos conectado (Tastytrade é corretora, não tem endpoint de fundamentos)	Confirmado via doc oficial Tastytrade
Consolidação de dataset	US_STOCKS_DATASET / SP500_DATASET	Dois catálogos parcialmente sobrepostos, risco de inconsistência	Ciclo 5
Histórico git paralelo	commits "Fatia 1-4"	Trabalho de remediação desconhecido, técnica divergente (usa provenance.ts, que nunca foi ligado)	git log desta sessão

2. Frente 1 — Dados reais de opções (em andamento)

Esta é a frente que gerou o achado da BAC e que você aprovou iniciar. Tem 4 etapas; a 1ª está feita.

1a. `getOptionChain(symbol)` — CONCLUÍDO E VALIDADO

Busca `GET /option-chains/{symbol}/nested` real, cacheado 5 min, retorna `null` (nunca fabrica) se a API falhar ou o formato vier inesperado. Testado ao vivo contra BAC: 19 vencimentos reais, strikes inteiros reais.

1b. Substituir a seleção sintética de strike/DTE — PRÓXIMO PASSO PROPOSTO

Mudança de contrato: `volatilityEngine.evaluate()` passa a exigir `chain: OptionChainResult | null` além do `input` atual.

- **Sem chain real** (`null`): a função retorna um resultado degradado explícito — sem `legs`, com um campo tipo `dataStatus: 'NO_REAL_CHAIN'` — e o componente que chama exibe “sem cadeia de opções real disponível para este ativo agora”, nunca cai de volta para strike/DTE calculado.
- **Com chain real**: duas funções novas substituem a lógica sintética:
 - `pickExpiration(chain, targetDteMin, targetDteMax)`: escolhe o vencimento real (`daysToExpiration` de verdade) mais próximo do centro da faixa-alvo da estratégia (hoje fixo em 30 ou 35; passa a ser uma faixa, ex. 30-45 DTE, com o vencimento real mais próximo do meio).
 - `pickStrikeNear(expiration.strikes, targetValue)`: acha o strike real mais próximo de um valor-alvo (put wall, call wall, spot) — substitui todo `Math.floor(x/step)*step` e `Math.ceil(x/step)*step` do arquivo atual.
- Efeito colateral necessário: a largura da trava (`width`, `putWingWidth`, `callWingWidth`) deixa de ser `step` fixo e passa a ser a diferença real entre strikes consecutivos disponíveis na chain — que pode variar (BAC mostrou \$45→\$49, um gap de \$4, não um `step` uniforme).

1c. Ponto de decisão que preciso da sua palavra antes de seguir: precificação da perna

Mesmo depois de 1b, o `midPrice` e o `delta` de cada perna ainda vêm do BSM com multiplicadores de IV inventados (`volDecimal * 1.08`, `* 1.03`, `* 0.98` ...) — não de uma cotação real. Isso é uma segunda fabricação, estrutural, dentro do mesmo arquivo, e seu mandato (“só dados reais”) não distingue strike de preço.

Duas rotas, com trade-off real:

Rota A — BSM sobre strike/DTE reais, rotulado como modelo. Mantém o motor de precificação atual (que já está provado correto matematicamente), aplicado sobre strikes e vencimentos agora reais. Mais rápido de entregar, sem chamada de API adicional. Honestidade exige rotular explicitamente “prêmio estimado por modelo BSM”, nunca “prêmio real” — e isso ainda deixa uma diferença potencial relevante entre o prêmio modelado e o prêmio realmente negociável no book (spread bid/ask real, skew real por strike).

Rota B — Cotação real por contrato. Busca `GET /market-data/by-type` usando os `callStreamerSymbol / putStreamerSymbol` que o `getOptionChain()` já captura, trazendo bid/ask/mid e delta reais de mercado. Resolve o problema por completo, mas é escopo novo: mais uma chamada de API por avaliação (latência), mais um ponto de falha (o preço pode não estar disponível fora do horário de pregão ou para strikes ilíquidos), e precisa de uma decisão de fallback quando a cotação não vier — que não pode ser “usa BSM em silêncio”, isso reintroduziria a mesma ambiguidade que estamos eliminando.

Minha recomendação: Rota B, com fallback declarado. Você pediu dado real, não dado plausível — e o BSM com multiplicador de IV arbitrário é exatamente a categoria de coisa que passou despercebida cinco ciclos seguidos. O comentário que já existe no código (`tastytrade-market.service.ts:195`, “numa próxima etapa”) mostra que isso já era esperado como próximo passo, não é escopo inventado agora. O custo (mais uma chamada de API, mais latência) é aceitável porque a avaliação de estratégia não é um endpoint de alta frequência — é acionado por interação do usuário, não em loop. Onde eu mitigaria o risco de indisponibilidade: se a cotação real vier vazia para uma perna específica, a UI mostra essa perna como “cotação indisponível — modelo BSM de referência: \$X,XX” com rótulo visualmente diferente (não mistura os dois na mesma cor/badge), em vez de bloquear a recomendação inteira. Isso assume o risco de UX (usuário vê uma mistura de real+modelo em casos raros) para mitigar o risco maior (voltar a apresentar preço fabricado como se fosse real).

Preciso da sua aprovação nesse ponto especificamente antes de escrever qualquer código de 1b/1c — é decisão de arquitetura, não implementação.

1d. Wiring nos consumidores

`QuoteView.tsx` e `VolatilityAnalystView.tsx` passam a buscar a chain (via nova rota de API, ex. `GET /api/market/option-chain?symbol=`, protegida por `applyApiGuard`, seguindo o mesmo padrão do `/api/health` já implementado) antes de chamar `evaluate()`, e tratam o estado “sem chain” na tela em vez de deixar o hook estourar ou renderizar vazio.

3. Frente 2 — Fundamentos reais de tickers US

Bloqueada desde antes desta sessão por falta de vendor. Nota relevante: você tem um conector FMP (Financial Modeling Prep) ativo nesta conversa de chat — isso não resolve automaticamente o backend do RADAR (o app roda no seu servidor/dev, não dentro desta sessão de chat), mas indica que você já tem acesso/chave FMP em algum nível. Vale confirmar se dá para reaproveitar a mesma chave como variável de ambiente do projeto (`FMP_API_KEY` em `.env.local`) em vez de contratar um vendor novo — antes de assumir que essa frente precisa de uma decisão comercial nova.

Enquanto não houver vendor: a via honesta imediata é a que já fizemos em `QuoteView.tsx` e `fundamentals/route.ts` — `currentRatio / ebitdaMargin / priceToBook` como `null`, nunca como constante. Isso ainda falta em `ai-consultant.ts:74-76` (achado C5-04) — é uma correção pequena, mecânica, idêntica à já feita, que pode entrar em qualquer ponto sem depender de decisão de vendor.

4. Frente 3 — Achados do Ciclo 5 ainda pendentes

Nenhum destes depende da Frente 1. Podem entrar em paralelo ou logo depois, custo baixo:

- `QuoteView.tsx:317-323` — checklist fixo (“SPOT: REAL (TASTYTRADE)”, “NET GEX POSITIVO”, “FED 4.50%”). Maior risco reputacional por menor esforço, segundo o próprio auditor Ciclo 5 — vira leitura dinâmica dos mesmos dados que já alimentam o resto da tela.
- `ai-consultant.ts:74-76` — replicar a correção → `null` já feita em outro arquivo.
- `provenance.ts` órfão — ou liga-se de verdade (badge MEDIDO/DERIVADO/ESTIMADO/SIMULADO exibido

na UI onde há dado real vs. modelo — que passa a fazer sentido de verdade depois da Frente 1c) ou remove-se do código para não deixar infraestrutura morta sugerindo rigor que não existe.

4. Consolidação `US_STOCKS_DATASET / SP500_DATASET` — depois das frentes acima, quando ficar claro quais campos numéricos migram para fonte real e quais seguem estáticos (nome, setor, categoria).

5. Frente 4 — Governança (não é código)

Dois pontos em aberto que não são bugs, são processo:

- **Commits “Fatia 1-4” desconhecidos** no histórico git, com abordagem técnica própria (usa `provenance.ts`) que nunca coordenou com este trabalho. Recomendo, antes de fechar o projeto como um todo (não bloqueia as frentes 1-3): rodar `git log --all --stat` numa sessão com você presente para entender se é outro agente/sessão sua, ou histórico de um colaborador, antes de decidir se aquele trabalho é descartado, mesclado ou motivo de investigação.
- **Arquivos de auditoria desaparecidos** (`AUDITORIA_RADAR_TASTYTRADE.md/pdf`, `AUDITORIA_RADAR_02_REAUDITORIA.md/pdf`, meu próprio `AUDITORIA_RADAR_04_CICLO4.md/pdf`) — sumiram do disco sem commit registrado. Não é urgente tecnicamente, mas é rastro de auditoria que você pode querer preservar. Recomendo simplesmente commitar os que restam (Ciclo 5 já está no disco) assim que o repositório for reorganizado, para não perder mais um.

6. Matriz de risco consolidada

Risco	Onde assumir	Onde mitigar
API Tastytrade fora do ar/latente no momento da avaliação	Assumir: recomendação fica indisponível por alguns segundos/minutos — aceitável para uma ferramenta de decisão, não de execução automática	Mitigar: cache de 5 min já implementado no <code>getOptionChain()</code> ; mensagem clara na UI, nunca fallback silencioso
Cotação real de perna específica indisponível (strike ilíquido, fora do pregão)	Assumir: mostrar BSM como referência explícita para aquela perna	Mitigar: rótulo visual diferenciado, nunca a mesma cor/selo de “real”
Latência extra da Rota B (1c) degradar a experiência	Assumir: endpoint não é de alta frequência, latência de 1-2s é tolerável numa tela de análise	Mitigar: loading state explícito na UI, sem isso vira sensação de trava
Retrabalho se a Frente 4 (Fatia 1-4) revelar solução já pronta em paralelo	Mitigar antes de investir mais: vale 15-20 min de investigação no <code>git log</code> antes de começar 1b, não depois	—
Escopo total (1+2+3) ser grande demais para revisar de uma vez	Mitigar: manter o padrão já validado nesta sessão — parte pequena, aprovação explícita, execução, verificação (<code>tsc + eslint + teste ao vivo</code> quando possível), próxima parte	—

7. Sequência recomendada

Meu julgamento, não é neutro — é uma recomendação:

1. **Frente 4, rápida (15-20 min, junto com você):** entender o `git log` das “Fatia 1-4” antes de mais nada. Risco de retrabalhar algo que já existe é o único item desta lista que fica mais caro quanto mais se adia.
2. **Decisão 1c** (Rota A vs B): preciso da sua palavra aqui antes de tocar em `volatility-engine.ts`, porque muda a superfície de código de forma diferente.
3. **Frente 1b+1c+1d completas:** é o núcleo do seu mandato “só dados reais” e a origem direta do achado da BAC — prioridade mais alta depois do item 1.
4. **Frente 3, itens 1 e 2** (checklist fixo + `ai-consultant.ts`): baratos, mecânicos, sem dependência — podem entrar em paralelo com o item 3 se quiser paralelizar, ou logo depois.
5. **Frente 2:** só depende de confirmar se a chave FMP do seu conector é reaproveitável; se não for, fica

em espera por decisão comercial sua, sem bloquear o resto.

6. **Frente 3, itens 3 e 4** (provenance.ts, consolidação de dataset): por último, porque só fazem sentido pleno depois que existir dado real de verdade para diferenciar visualmente (item 3 da Frente 3 depende conceitualmente da Frente 1c).
7. **Auditoria cega independente, sessão nova**, only depois de 1-4 completos — não antes, para não repetir o padrão de “auditoria de fórmula” sem grade real para comparar.

Meu ponto: não recomendo tentar as Frentes 1-4 em paralelo total. O motivo é o mesmo que já vimos nesta sessão — quanto mais partes abertas ao mesmo tempo, maior a chance de outra lacuna de categoria (não de fórmula) passar despercebida. Sequência com aprovação em cada parte, como já vínhamos fazendo, é mais lenta por hora mas mais barata no total, porque falha rápido e barato em vez de acumular cinco ciclos de auditoria para achar um erro visual óbvio.

O que preciso de você agora, especificamente: (a) aprovar ou ajustar a sequência acima, (b) decidir Rota A ou B no item 1c, e (c) confirmar se quer a checagem rápida do `git log` da Frente 4 antes de eu tocar em qualquer código.